

Learning Minimally-Congested Drive Times from Sparse Open Networks



A lightweight ML correction to traditional length/speed travel-time estimates

keywords: Travel Time Prediction • Open Data • Random Forest

PI: Adewumi Augustine Adepitan

Co-Author: Christopher Haruna, Morayo Ogunsina, Damilola Yussuf, and Ayooluwatomiwa Ajiboye

Submission ID 23

SMiLE 2026

The 2nd International Conference on Smart Mobility and Logistics Ecosystems

KFUPM, Dhahran, Saudi Arabia

Feb 8 - 11, 2026

Motivation

Why travel time matters

Planning decisions depend on it



Accessibility

Who can reach jobs, schools, healthcare within a time threshold?



Operations

Network performance, reliability, and route planning workflows



Equity

Bias can overstate access for some communities



**Better travel-time estimates
→ better decisions**

Problem

A practical gap in travel-time prediction

Between “accurate but expensive” and “simple but biased”

Our goal: middle ground



Data-intensive methods

- Massive GPS/probe traces
- Sensors + proprietary feeds
- Deep learning / complex models

Accurate

Hard to scale

Costly / black-box



Simplified planning methods

- Distance / speed limit
- Naïve traversal time
- Easy to implement

Fast & accessible

Biased low

Ignores delays

Baseline

Traditional baseline: “length ÷ speed”

Dijkstra minimizes traversal time using speed limits

Traversal time on each road segment

$$\text{time} = \text{length} / \text{speed_limit}$$

Route travel time = sum of segment times (Dijkstra chooses the minimum)



Fastest path (no delays)

What it misses (causes underestimation)



Signals / stop signs / crossings



Turning delays + intersection friction



Acceleration / deceleration effects

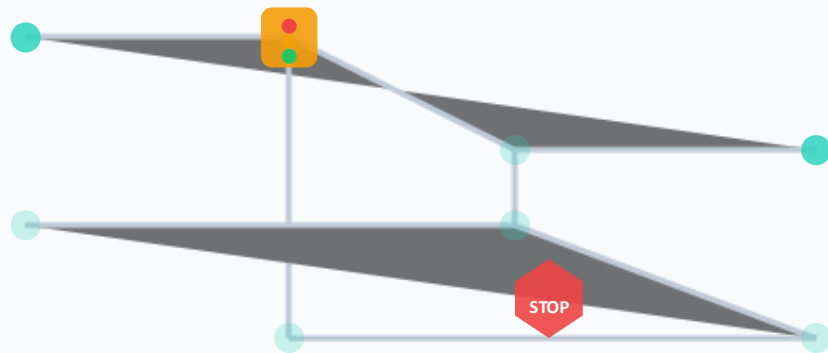
Result: systematic underprediction

Methodology

What “we consider in sparse data” here

Not dense traffic measurements — just a few basic, often-missing network attributes

We do have (open)



Most intersections are untagged → features are sparse

- Road lengths
- Speed limits
- Some control tags (signals, stops, crossings)
- Geometry to infer turning movements

We do NOT rely on

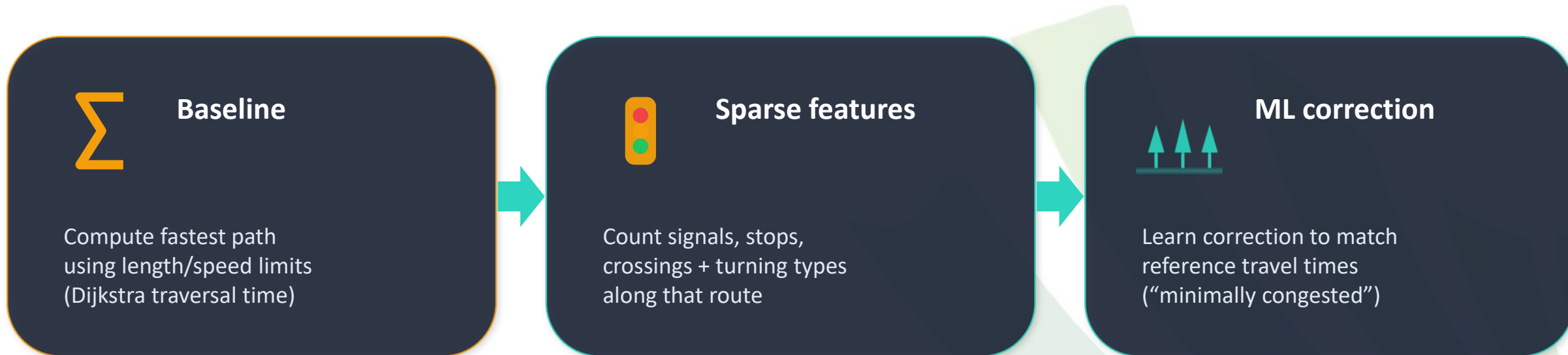
- Real-time congestion feeds
- Large GPS probe datasets
- Proprietary sensor networks
- Black-box travel-time APIs (except limited training labels)

**Key idea: learn a correction
from a *small* set of high-quality labels**

Core idea

We do not replace the baseline — we correct it

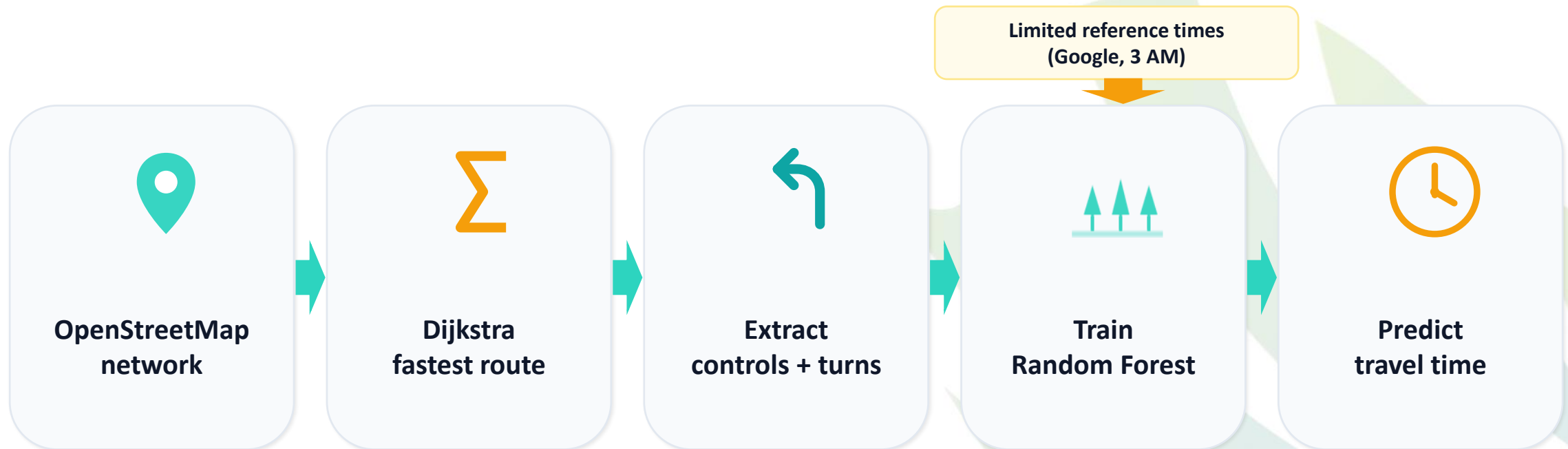
Use sparse operational signals (controls + turns) to learn a correction to traversal-time bias.



Workflow

End-to-end workflow

From open network data → baseline route → sparse features → trained correction model



Features

What the model “sees” for each trip

11 simple predictors (counts + baseline time)



Baseline

- Naïve travel time
- (Dijkstra traversal-time)



Traffic controls

- Signals
- Stop signs
- Crossings
- Give way
- Roundabouts



Turning movements

- Left
- Slight left
- Right
- Slight right
- U-turn

Important: we do NOT assume delays — the model learns them from data.

How the model learns (no hand-tuned delay penalties)

Each OD trip becomes a training row: features from the open network → target from limited reference times.

Training example (one trip)

Inputs (X)

t_naive
#signals
#stops
#turns...

Target (y)

**Google travel
time (3 AM)**

Train/test split: 80/20 + 5-fold CV

Random Forest “correction”



400 trees (ensemble)
Average their predictions
→ robust to sparse features



Corrected travel time

Metrics

What is “prediction error”?

A simple way to quantify how far a prediction is from reality



Actual (reference)
travel time

20 min



Error = 4 min

Predicted
travel time

16 min



Percentage error = $| \text{Pred} - \text{Actual} | / \text{Actual}$
MAPE = average percentage error across all trips

Model Performance Results

Model	MAPE (%)	MAE (s)	Bias δ (s)	R ²
Naïve	21.2	184	-182.9	0.74
RF (Ours)	8.41	75	0.38	0.93
GB	7.86	72	-19.2	0.93
DT	9.00	80	0.13	0.93
AdaBoost	8.20	74	-9.76	0.93

MAPE
21.2% →
8.4%
 ↓ 60% error

Bias (δ)
-183s →
0s
 Unbiased

R²
0.74 →
0.93
 High fit

MAE
184s →
75s
 2.4× better

Why Random Forest Was Selected

Bias–Variance Balance

- Gradient Boosting: low error, but under-predicts
- Decision Tree: unbiased, high variance
- Random Forest: stable and accurate
- Best overall engineering performance

Feature Importance Insights

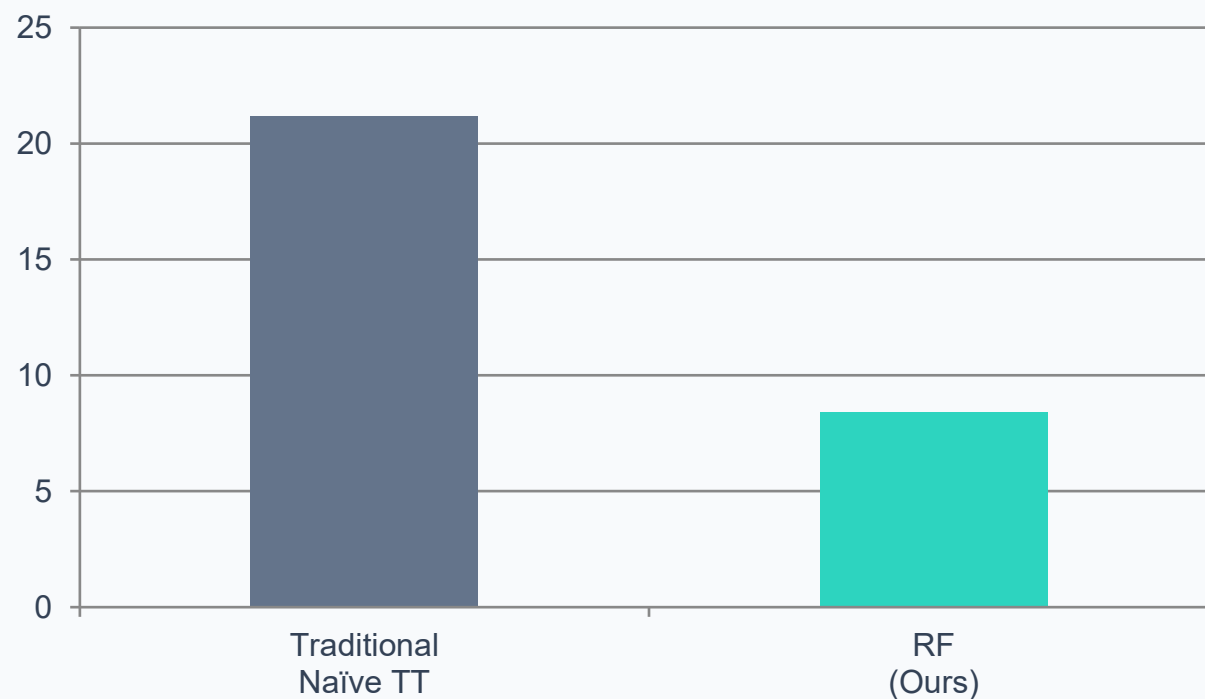
- Baseline travel time explains ~68%
- Signals & stops are key delay drivers
- Turning movements ≈ 19% contribution
- Sparse open data still improves prediction

Results

Results: large error reduction with minimal bias

Out-of-sample performance (test set): Traditional traversal-time baseline vs Random Forest correction

Mean Absolute Percentage Error (MAPE)



Key takeaways

21% → 8%

Typical percentage error
reduced by >60%

Bias (mean δ):

Naïve = -182.9 s

RF = +0.38 s (ns)

Explained variance:

R^2 improves from 0.74 → 0.93

Why this matters in practice

A “middle ground” tool for metro-scale analysis



Scalable

Runs on standard hardware
for large urban networks



Open-data driven

Uses OpenStreetMap
+ sparse operational tags



Defensible accuracy

Cuts typical error
by more than half



Transparent

No black-box traffic feed
required for predictions

Key takeaways

- Traversal-time baselines are useful but systematically optimistic in cities.
- Sparse open data still contain operational signals (controls + turns).
- A lightweight Random Forest correction can achieve strong accuracy without congestion feeds.

Thank you — Questions?